

ACA Lab–6-7: Shared Memory Multiprogramming–II

Topic: Self-Scheduling

AIM: Programs using self-scheduling and locks in parallel processing.

EXERCISES:-

1. Complete Programs 5 to 8 of Loop Splitting from Previous Lab (Lab-5), which includes Matrix Addition, Multiplication, and Factorial of the given number.

=> Done in Lab 5.

2. In the header.h file created in LAB 5, include the following functions:

- **spin lock init()**
- **spin lock()**
- **spin unlock()**

=>

```
#include <sys/wait.h>
#include <unistd.h>
#include <sys/shm.h>
#include <stdlib.h>
#include <sys/sem.h>
```

```
/* Create multiple processes */
```

```
int process_fork(int nproc) {
    int j;
    for (j = 1; j < nproc; j++) {
        if (fork() == 0)
            return j;
    }
    return 0;
}
```

```
/* Join processes */
```

```
void process_join(int nproc, int id) {
    int i;
    if (id == 0) {
        for (i = 1; i < nproc; i++)
            wait(0);
    } else {
        exit(0);
    }
}
```

```

/* Shared memory creation */
char* shared(int size, int *shmid) {
    *shmid = shmget(IPC_PRIVATE, size, 0666 | IPC_CREAT);
    return (char *) shmat(*shmid, 0, 0);
}

/* Initialize spin lock */
void spin_lock_init(int *lock, int *condition) {
    int control;

    *lock = semget(IPC_PRIVATE, 1, 0666 | IPC_CREAT);

    if (*condition == 1)
        control = 0;
    else
        control = 1;

    semctl(*lock, 0, SETVAL, control);
}

/* Acquire spin lock */
void spin_lock(int *lock) {
    struct sembuf operations;

    operations.sem_num = 0;
    operations.sem_op = -1;
    operations.sem_flg = 0;

    semop(*lock, &operations, 1);
}

/* Release spin lock */
void spin_unlock(int *lock) {
    struct sembuf operations;

    operations.sem_num = 0;
    operations.sem_op = 1;
    operations.sem_flg = 0;

    semop(*lock, &operations, 1);
}

```

3. Write a program to add a constant to an array using self-scheduling.

=>

```
#include <stdio.h>
#include "header.h"

int main() {
    int *a, *next_index;
    int i, id;
    int k = 5, nproc = 100, shmid, n = 10000;

    /* Shared variable for next index */
    next_index = (int *) shared(sizeof(int), &shmid);
    *next_index = 0;

    /* Shared array */
    a = (int *) shared(sizeof(int) * n, &shmid);
    for (i = 0; i < n; i++)
        *(a + i) = 0;

    /* Create processes */
    id = process_fork(nproc);

    /* Critical section (without lock) */
    while (1) {
        i = *next_index;
        *next_index = *next_index + 1;

        if (i < n)
            *(a + i) = *(a + i) + k;
        else
            break;
    }

    /* Join processes */
    process_join(nproc, id);
}
```

Output:-



4. Write a parallel program to implement Exercise 3 using locking.

=>

```
#include <stdio.h>
#include "header.h"
```

```
int main() {
    int *a, *next_index, *lock;
    int i, id;
    int k = 5, nproc = 100, shmid, n = 10000;
    int locked = 1, unlocked = 0;

    /* Shared variable for index */
    next_index = (int *) shared(sizeof(int), &shmid);
    *next_index = 0;

    /* Shared array */
    a = (int *) shared(sizeof(int) * n, &shmid);
    for (i = 0; i < n; i++)
        *(a + i) = 0;

    /* Shared spin lock */
}
```

```

lock = (int *) shared(sizeof(int), &shmid);
spin_lock_init(lock, &unlocked);

/* Create processes */
id = process_fork(nproc);

/* Critical section protected by spin lock */
while (1) {
    spin_lock(lock);

    i = *next_index;
    *next_index = *next_index + 1;

    spin_unlock(lock);

    if (i < n)
        *(a + i) = *(a + i) + k;
    else
        break;
}

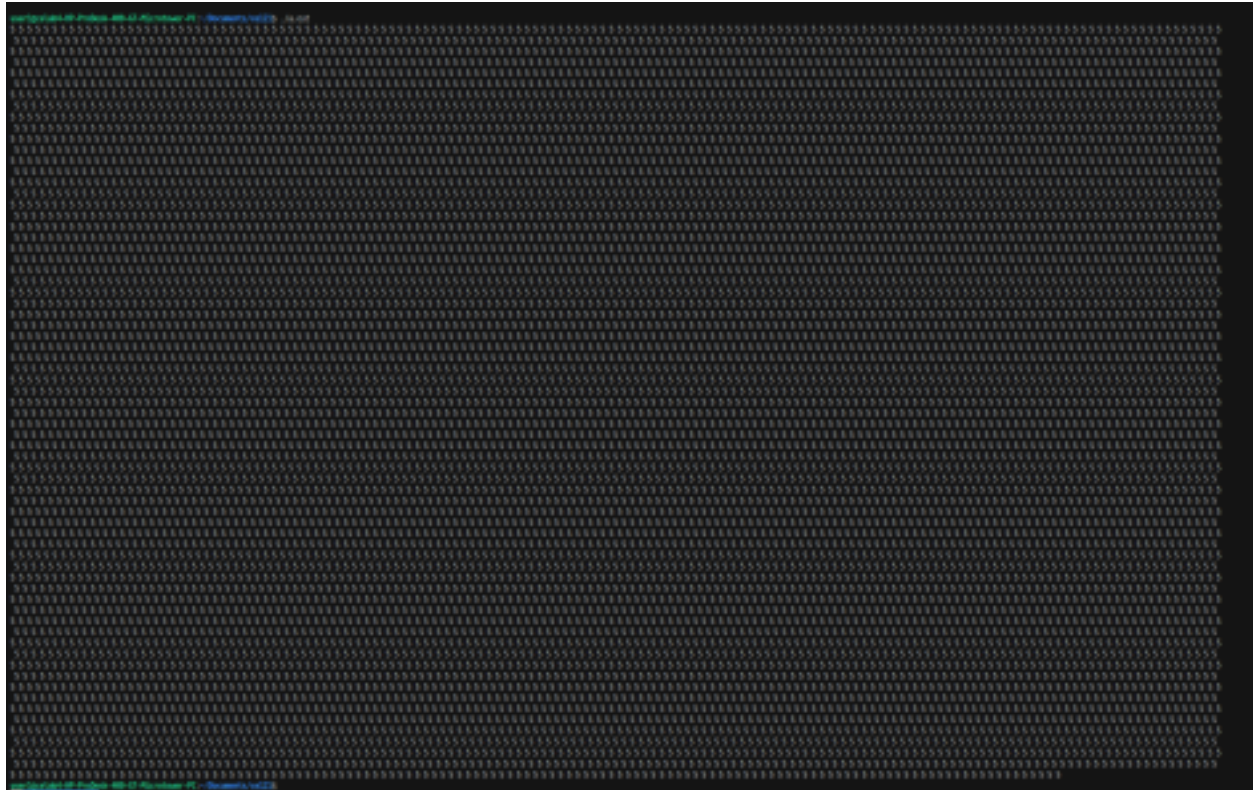
/* Join processes */
process_join(nproc, id);

/* Print result */
for (i = 0; i < n; i++)
    printf("%d ", *(a + i));

return 0;
}

```

Output:-



5. Write a parallel program to calculate the sum of numbers from 1 to n using m processes.

=>

```
#include <stdio.h>
```

```
#include "header.h"
```

```
int main() {
```

```
    int sum = 0, *final_sum;
```

```
    int i, id, k = 5, nproc, shmid, N;
```

```
    int unlocked = 0;
```

```
    int *lock1, *lock2, *index;
```

```
    /* Shared final sum */
```

```
    final_sum = (int *) shared(sizeof(int), &i);
```

```
    *final_sum = 0;
```

```
    /* Shared locks and index */
```

```
    lock1 = (int *) shared(sizeof(int), &shmid);
```

```

lock2 = (int *) shared(sizeof(int), &shmid);
index = (int *) shared(sizeof(int), &shmid);
*index = 1;
printf("Enter number : ");
scanf("%d", &N);
printf("Enter number of Processes : ");
scanf("%d", &nproc);
/* Initialize locks */
spin_lock_init(lock1, &unlocked);
spin_lock_init(lock2, &unlocked);
/* Create processes */
id = process_fork(nproc);
/* Each process computes partial sum */
while (1) {
    spin_lock(lock1);
    i = *index;
    *index = *index + 1;
    spin_unlock(lock1);
    if (i <= N)
        sum = sum + i;
    else
        break;
}
/* Update final sum safely */
spin_lock(lock2);
*final_sum = sum + *final_sum;

```

```

spin_unlock(lock2);

/* Join processes */

process_join(nproc, id);

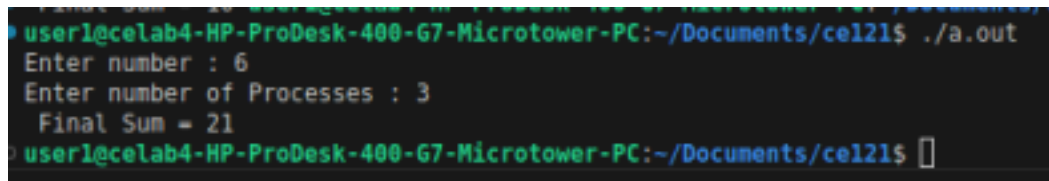
printf("Final Sum: %d\n", *final_sum);

return 0;

}

```

Output:-



```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Enter number : 6
Enter number of Processes : 3
Final Sum = 21
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$

```

6. Write a parallel program to calculate the sum of an array of n elements using self-scheduling.

=>

```

#include <stdio.h>

#include "header.h"

int main() {
    int sum = 0, *final_sum;
    int id, a[10], i = 0, nproc = 4;
    int unlocked = 0;
    int *lock1, *lock2;
    int shmid, *index;

    /* Shared variables */

    final_sum = (int *) shared(sizeof(int), &shmid);
    *final_sum = 0;

    lock1 = (int *) shared(sizeof(int), &shmid);
    lock2 = (int *) shared(sizeof(int), &shmid);
    index = (int *) shared(sizeof(int), &shmid);
}

```



```

*index = 0;

/* Input array */
for (i = 0; i < 10; i++) {
    printf("Enter a[%d] values : ", i + 1);
    scanf("%d", &a[i]);
}

/* Initialize locks */
spin_lock_init(&lock1, &unlocked);
spin_lock_init(&lock2, &unlocked);

/* Create processes */
id = process_fork(nproc);

/* Parallel sum calculation */
while (1) {
    spin_lock(&lock1);
    i = *index;
    *index = *index + 1;
    spin_unlock(&lock1);
    if (i < 10)
        sum = sum + a[i];
    else
        break;
}

/* Combine partial sums safely */
spin_lock(&lock2);
*final_sum = sum + *final_sum;
spin_unlock(&lock2);

/* Join processes */

```

```

process_join(nproc, id);

/* Output */

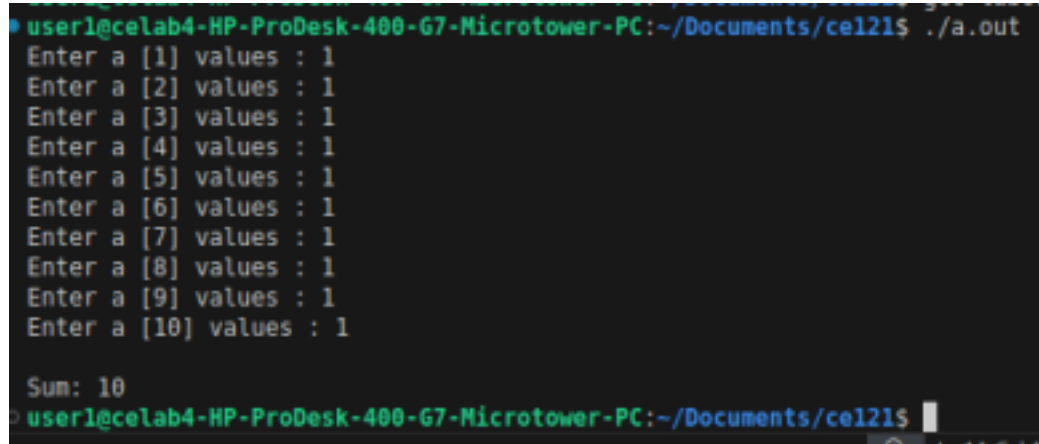
printf("\nSum: %d\n", *final_sum);

return 0;

}

```

Output:-



```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Enter a [1] values : 1
Enter a [2] values : 1
Enter a [3] values : 1
Enter a [4] values : 1
Enter a [5] values : 1
Enter a [6] values : 1
Enter a [7] values : 1
Enter a [8] values : 1
Enter a [9] values : 1
Enter a [10] values : 1

Sun: 10
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$

```

7. Implementation of histogram in different ways:

(a) Create a histogram using self-scheduling.

=>

```

#include "header.h"
#include <stdlib.h>
#include <stdio.h>

#define arrSize 15

int main() {
    int a[arrSize], i;
    int *index, *histogram;
    int NoOfBins = 5, binsize;
    int *lock1, *lock2;
    int unlocked = 0, locked = 1;
    int shmidindex, shmidlock1, shmidlock2, shmidhist;
    int id, nproc, bin, amin, amax;

    /* Input array */
    for (i = 0; i < arrSize; i++) {
        printf("Enter a[%d] : ", i);
        scanf("%d", &a[i]);
    }
}

```

```

}

/* Find min and max */
amin = a[0];
amax = a[0];

for (i = 1; i < arrSize; i++) {
    if (amin > a[i])
        amin = a[i];
    if (amax < a[i])
        amax = a[i];
}

/* Calculate bin size */
binsize = (amax - amin) / NoOfBins;

/* Shared memory allocation */
index = (int *) shared(sizeof(int), &shmidindex);
lock1 = (int *) shared(sizeof(int), &shmidlock1);
lock2 = (int *) shared(sizeof(int), &shmidlock2);
histogram = (int *) shared(sizeof(int) * NoOfBins, &shmidhist);

printf("\nBin Size : %d", binsize);
printf("\nNo. Of Bins : %d\n", NoOfBins);

/* Initialize locks */
spin_lock_init(lock1, &unlocked);
spin_lock_init(lock2, &unlocked);

*index = 0;

/* Initialize histogram */
for (i = 0; i < NoOfBins; i++)
    *(histogram + i) = 0;

/* Create processes */
nproc = NoOfBins;
id = process_fork(nproc);

/* Histogram calculation */
while (1) {

```

```

    spin_lock(lock1);

    i = *index;
    *index = *index + 1;

    spin_unlock(lock1);

    if (i >= arrSize)
        break;

    bin = abs((a[i] - amin) / binsize);
    if (bin >= NoOfBins)
        bin = NoOfBins - 1;

    printf("Number %d is : %d\tBin : %d\n", i, a[i], bin);

    spin_lock(lock2);
    *(histogram + bin) += 1;
    spin_unlock(lock2);
}

/* Join processes */
process_join(nproc, id);

/* Print histogram */
for (i = 0; i < NoOfBins; i++)
    printf("No of Items in Bin (%d): %d\n", i, *(histogram + i));

return 0;
}

```

Output:-

```
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
enter a [0] :0
enter a [1] :11
enter a [2] :12
enter a [3] :19
enter a [4] :22
enter a [5] :23
enter a [6] :24
enter a [7] :25
enter a [8] :27
enter a [9] :29
enter a [10] :35
enter a [11] :36
enter a [12] :37
enter a [13] :45
enter a [14] :50

Bin Size : 10
No. Of Bins : 5
Number 0 is : 0 Bin : 0
Number 1 is : 11 Bin : 1
Number 2 is : 12 Bin : 1
Number 3 is : 19 Bin : 1
Number 5 is : 23 Bin : 2
Number 6 is : 24 Bin : 2
Number 7 is : 25 Bin : 2
Number 8 is : 27 Bin : 2
Number 9 is : 29 Bin : 2
Number 10 is : 35 Bin : 3
Number 4 is : 22 Bin : 2
Number 11 is : 36 Bin : 3
Number 12 is : 37 Bin : 3
Number 13 is : 45 Bin : 4
Number 14 is : 50 Bin : 4
No of Items in Bin (0): 1
No of Items in Bin (1): 3
No of Items in Bin (2): 6
No of Items in Bin (3): 3
No of Items in Bin (4): 2
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$
```

(b) Implement a histogram using loop splitting.

=>

```
#include <stdio.h>
#include "header.h"
```

```
#define arrSize 15
```

```
int main() {
    int a[arrSize];
    int NoOfBins = 5, binsize;
    int *histogram;
```

```

int *lock, unlocked = 0, locked = 1;
int shmidlock, shmidhist;
int id, nproc, bin, amin, amax, i;

/* Input array */
for (i = 0; i < arrSize; i++) {
    printf("Enter a[%d] : ", i);
    scanf("%d", &a[i]);
}

/* Find min and max */
amin = a[0];
amax = a[0];

for (i = 1; i < arrSize; i++) {
    if (amin > a[i])
        amin = a[i];
    if (amax < a[i])
        amax = a[i];
}

/* Calculate bin size */
if ((amax - amin) == 0)
    binsize = 1;
else
    binsize = (amax - amin) / NoOfBins;

/* Shared memory allocation */
lock = (int *) shared(sizeof(int) * NoOfBins, &shmidlock);
histogram = (int *) shared(sizeof(int) * NoOfBins, &shmidhist);

printf("\nBin Size : %d", binsize);
printf("\nNo. Of Bins : %d\n", NoOfBins);

/* Initialize locks (one lock per bin) */
for (i = 0; i < NoOfBins; i++)
    spin_lock_init(lock + i, &unlocked);

/* Initialize histogram */
for (i = 0; i < NoOfBins; i++)
    *(histogram + i) = 0;

```

```

printf("\n");

/* Create processes */
nproc = NoOfBins;
id = process_fork(nproc);

/* Parallel histogram computation */
for (i = id; i < arrSize; i = i + nproc) {
    bin = abs(a[i] - amin) / binsize;
    if (bin >= NoOfBins)
        bin = NoOfBins - 1;

    printf("Process(%d): Number[%d] = %d \t Bin: %d\n",
        id, i, a[i], bin);

    spin_lock(lock + bin);
    *(histogram + bin) += 1;
    spin_unlock(lock + bin);
}

printf("\n");

/* Join processes */
process_join(NoOfBins, id);

/* Print histogram */
for (i = 0; i < NoOfBins; i++)
    printf("No Of Items in Bin (%d) is %d\n",
        i, *(histogram + i));

return 0;
}
Output:-

```

```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
enter a[0] : 0
enter a[1] : 11
enter a[2] : 12
enter a[3] : 19
enter a[4] : 22
enter a[5] : 23
enter a[6] : 24
enter a[7] : 25
enter a[8] : 27
enter a[9] : 29
enter a[10] : 35
enter a[11] : 36
enter a[12] : 37
enter a[13] : 45
enter a[14] : 50

Bin Size : 10
No. Of Bins : 5

Process( 1 ):Number [1] is 11    Bin: 1
Process( 1 ):Number [6] is 24    Bin: 2
Process( 1 ):Number [11] is 36   Bin: 3

Process( 2 ):Number [2] is 12    Bin: 1
Process( 2 ):Number [7] is 25    Bin: 2
Process( 2 ):Number [12] is 37   Bin: 3

Process( 3 ):Number [3] is 19    Bin: 1
Process( 0 ):Number [0] is 0     Bin: 0
Process( 3 ):Number [8] is 27    Bin: 2
Process( 3 ):Number [13] is 45   Bin: 4
Process( 0 ):Number [5] is 23    Bin: 2
Process( 0 ):Number [10] is 35   Bin: 3

Process( 4 ):Number [4] is 22    Bin: 2
Process( 4 ):Number [9] is 29    Bin: 2
Process( 4 ):Number [14] is 50   Bin: 4

No Of Items in Bin (0) is 1
No Of Items in Bin (1) is 3
No Of Items in Bin (2) is 6
No Of Items in Bin (3) is 3
No Of Items in Bin (4) is 2
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$

```

(c) Implement a histogram using partial histograms.

=>

```

#include <stdio.h>
#include <stdlib.h>
#include "header.h"

```

```

#define arrSize 15

```

```

int main() {
    int a[arrSize];
    int NoOfBins, binsize;

```



```

int *histogram;
int *lock, unlocked = 0, locked = 1;
int shmlock, shmhist;
int id, nproc, bin, amin, amax, i;

/* Input number of bins */
printf("Enter No Of Bins : ");
scanf("%d", &NoOfBins);

int parhist[NoOfBins];

/* Input array */
for (i = 0; i < arrSize; i++) {
    printf("Enter a[%d] : ", i);
    scanf("%d", &a[i]);
}

/* Find min and max */
amin = a[0];
amax = a[0];

for (i = 1; i < arrSize; i++) {
    if (amin > a[i])
        amin = a[i];
    if (amax < a[i])
        amax = a[i];
}

/* Calculate bin size */
if ((amax - amin) > 0)
    binsize = (amax - amin) / NoOfBins;
else
    binsize = 1;

/* Shared memory allocation */
lock = (int *) shared(sizeof(int) * NoOfBins,
&shmlock);
histogram = (int *) shared(sizeof(int) *
NoOfBins, &shmhist);

printf("\nBin Size : %d", binsize);
printf("\nNo. Of Bins : %d\n", NoOfBins);

```

```

/* Initialize locks */
for (i = 0; i < NoOfBins; i++)
    spin_lock_init(lock + i, &unlocked);

/* Initialize global histogram */
for (i = 0; i < NoOfBins; i++)
    *(histogram + i) = 0;

printf("\n");

/* Create processes */
nproc = NoOfBins;
id = process_fork(nproc);

/* Initialize private histogram */
for (i = 0; i < NoOfBins; i++)
    parhist[i] = 0;

/* Each process computes its private histogram
*/
for (i = id; i < arrSize; i = i + nproc) {
    bin = abs(a[i] - amin) / binsize;
    if (bin >= NoOfBins)
        bin = NoOfBins - 1;

    printf("Process(%d): Number[%d] = %d \t
Bin: %d\n",
        id, i, a[i], bin);

    parhist[bin] += 1;
}

/* Merge private histograms into global
histogram */
for (i = 0; i < NoOfBins; i++) {
    spin_lock(lock + i);
    *(histogram + i) += parhist[i];
    spin_unlock(lock + i);
}

printf("\n");

```

```
/* Join processes */
process_join(nproc, id);

/* Print final histogram */
for (i = 0; i < NoOfBins; i++)
    printf("No Of Items in Bin (%d) is %d\n",
        i, *(histogram + i));

return 0;
}
```

Output:-

```
user1@celab5-Veriton-M200-P500:~/Downloads/cel2932_cn/RS-232$ ./a.  
Enter No Of Bins : 5  
enter a[0] : 0  
enter a[1] : 11  
enter a[2] : 12  
enter a[3] : 19  
enter a[4] : 22  
enter a[5] : 23  
enter a[6] : 24  
enter a[7] : 25  
enter a[8] : 27  
enter a[9] : 29  
enter a[10] : 35  
enter a[11] : 36  
enter a[12] : 37  
enter a[13] : 45  
enter a[14] : 50  
  
Bin Size : 10  
No. Of Bins : 5  
  
Process(1):Number[1] is 11      Bin: 1  
Process(1):Number[6] is 24      Bin: 2  
Process(1):Number[11] is 36     Bin: 3  
  
Process(2):Number[2] is 12      Bin: 1  
Process(2):Number[7] is 25      Bin: 2  
Process(2):Number[12] is 37     Bin: 3  
  
Process(3):Number[3] is 19      Bin: 1  
Process(3):Number[8] is 27      Bin: 2  
Process(3):Number[13] is 45     Bin: 4  
  
Process(0):Number[0] is 0       Bin: 0  
Process(0):Number[5] is 23      Bin: 2  
Process(0):Number[10] is 35     Bin: 3  
  
Process(4):Number[4] is 22      Bin: 2  
Process(4):Number[9] is 29      Bin: 2  
Process(4):Number[14] is 50     Bin: 4  
  
No Of Items in Bin (0) is 1  
No Of Items in Bin (1) is 3  
No Of Items in Bin (2) is 6  
No Of Items in Bin (3) is 3  
No Of Items in Bin (4) is 2  
user1@celab5-Veriton-M200-P500:~/Downloads/cel2932_cn/RS-232$ █
```